Data Structures in Python: Lists and Tuples

Comprehensive Lecture & Code Walkthrough (62 Slides)

Touchpad Robotics & Artificial Intelligence-X

Computer Science Department

July 29, 2026

Table of Contents

1. Python Lists - Fundamentals
2. List Operations & Methods
3. Python Tuples - Fundamentals
4. Comparison & Advanced Operations
5. Summary & Final Review

1.1 Introduction to Python Lists

What is a List?

A **List** is a collection of **heterogeneous data** arranged in an ordered sequence.

- Values can be of any data type: `int`, `float`, `string`, objects, or nested lists.
- Each individual value inside a list is called an **element**.
- The total count of elements forms the **length** of the list.
- Elements are separated by **commas** and enclosed in **square brackets** `[]`.

1.2 Key Features of Lists

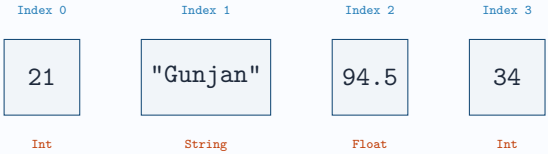
Core Characteristics

- **Mutable**: Content can be modified, added, or deleted.
- **Ordered**: Elements maintain their sequence.
- **Indexed**: Each value is accessed via an index number.

Data Storage

- **Heterogeneous**: Stores multiple data types simultaneously.
- **Syntax**: `[val1, val2, val3]`.
- **Dynamic**: Resizable during program execution.

1.3 Conceptual Memory Model of a List



Heterogeneous Memory Alignment

Notice how integer, string, and float values reside sequentially inside the same contiguous list structure `students = [21, "Gunjan", 94.5, 34]`.

1.4 Creating Lists: Overview

Lists can be instantiated in various ways depending on requirements:

1. **Empty List Creation**
2. **Single Value & Homogeneous Lists**
3. **Heterogeneous (Mixed) Lists**
4. **Nested Lists**
5. **Repeated Values List**
6. **Type Conversion via `list()` Function**
7. **Dynamic User Input List**

1.5 Creating Empty and Single-Value Lists

```
1 # Creating an Empty List
2 list1 = []
3 # Creating a List with a Single Value
4 list2 = [56]
```

Key Takeaway

- An empty list [] has a length of 0.
- Unlike tuples, a single-value list does **not** require a trailing comma.

1.6 Creating Homogeneous Lists

```
# List of Friends (Strings)
friends = ["Saisha", "Advika", "Arshia"]
# List of Marks (Integers)
marks = [92, 91, 89, 95, 93]
```

Usage Note

Homogeneous lists store uniform data types and are heavily used for standard statistical and sequential calculations.

1.7 Creating Heterogeneous (Mixed) Lists

```
1 # Mixed list containing Roll No, Name, Percentage
2 students = [21, "Gunjan", 94.5, 34, "Rohit", 91.3]
```

Data Structure Insights

- `students[0]` → `21` (`int`)
- `students[1]` → `"Gunjan"` (`str`)
- `students[2]` → `94.5` (`float`)

1.8 Creating Nested Lists

```
# List containing another list as an element
section = ["A", 25, "English", ["B", 34], ["C", 37], 56.7]
```

Accessing Sub-Lists

- `section[3]` returns `["B", 34]`.
- `section[3][0]` returns `"B"`.

1.9 Creating Lists with Repeated Values

```
# Repetition Operator (*)
L1 = [10] * 5
print(L1)
# Output: [10, 10, 10, 10, 10]
```

Mechanism

The $*$ operator multiplies the list items by the specified integer factor, duplicating the initial sequence.

1.10 List Creation via `list()` Constructor

```
# Converting String to List
alpha = list("hello")
print(alpha)
# Output: ['h', 'e', 'l', 'l', 'o']
```

Conversion Rule

The `list()` function breaks down any iterable sequence (like strings, tuples, sets) into individual elements.

1.11 Dynamic User Input for Lists

```
# Reading user input directly as a list
L1 = list(input("Enter numbers: "))
```

Behavior Warning

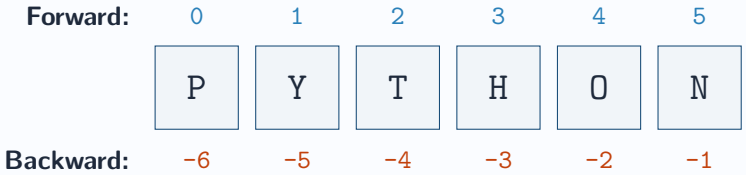
If the user enters 123, `list(input())` converts the input string `"123"` into `['1', '2', '3']`.

1.12 List Indexing Architecture

Python supports dual-indexing for all sequence types:

- **Forward Indexing (Positive)**: Starts from `0` at the leftmost element up to `length - 1`.
- **Backward Indexing (Negative)**: Starts from `-1` at the rightmost element down to `-length`.

1.13 Diagrammatic Representation of Indexing



List Definition

Given `L1 = ['P', 'Y', 'T', 'H', 'O', 'N']`, length is 6.

1.14 Accessing Elements Code Examples

Given: `L1 = ['P', 'Y', 'T', 'H', 'O', 'N']`

```
print(L1[0]) # Output: 'P'
print(L1[3]) # Output: 'H'
print(L1[-3]) # Output: 'H'
```

```
1 print(L1[-1]) # Output: 'N'
2 print(L1) # Output: ['P', 'Y', 'T', 'H', '
      O', 'N']
3 print(L1[0], L1[4]) # Output: P O
```

Syntax Rule

Access syntax: `listname[index_number]`.

1.15 Task Problem Statement

Problem Challenge

Given the list:

```
word = ['E', 'D', 'U', 'C', 'A', 'T', 'I', 'O', 'N']
```

Determine the exact output of the following slice operations:

1. `print(word[:1:-2])`
2. `print(word[-1:])`
3. `print(word[:-1])`

1.16 Task Solution & Step-by-Step Analysis

```
word = ['E', 'D', 'U', 'C', 'A', 'T', 'I', 'O', 'N']
# 1. word[:1:-2] -> Starts at end, steps backwards by 2 until index 1
# Output: ['N', 'I', 'A', 'U']
# 2. word[-1:] -> Slices from index -1 to the end
# Output: ['N']
# 3. word[:-1] -> Slices from start up to index -1 (excluded)
# Output: ['E', 'D', 'U', 'C', 'A', 'T', 'I', 'O']
```

2.1 Introduction to List Manipulation

Overview

List operations involve adding, removing, searching, and changing elements within a list.

- **Adding Elements**: `append()`, `insert()`, `extend()`
- **Modifying Elements**: Direct index assignment.
- **Sorting Elements**: `sort()`
- **Searching Elements**: Loops, `in` operator, `count()`, `index()`

2.2 Functions for Adding Elements

Function	Description	Length Change
`append(item)`	Adds a single item at the end	$+1$
`insert(idx, item)`	Inserts item at specified index	$+1$
`extend(iterable)`	Appends multiple items from iterable	$+N$

2.3 The append() Function

Definition & Syntax

Syntax: `list.append(item)`

- Appends a **single element** to the end of the list.
- The item can be of any data type (number, string, list, etc.).
- Increases the list length by exactly 1.

2.4 append() Code Examples (Basic)

```
# Example 1: Appending a string
fruits = ["apple", "banana", "cherries"]
fruits.append("melon")
print(fruits)
# Output: ['apple', 'banana', 'cherries', 'melon']
# Example 2: Sequential appends
cars = ["Maruti", "Ford"]
cars.append("Honda")
cars.append("Hyundai")
print(cars)
# Output: ['Maruti', 'Ford', 'Honda', 'Hyundai']
```

2.5 append() in Loops and Nested Append

```
# Example 3: Adding user input via loop
friends = []
for i in range(5):
    nm = input("Enter Friend Name: ")
    friends.append(nm)
print(friends)
# Example 4: Appending a list (Nested List result)
pets = ['cat', 'dog', 'rabbit']
wild = ['tiger', 'lion']
pets.append(wild)
print(pets)
# Output: ['cat', 'dog', 'rabbit', ['tiger', 'lion']]
```

2.6 The `insert()` Function

Definition & Syntax

Syntax: `list.insert(index, value)`

- Inserts a single element at a **specific position**.
- Shifts existing elements to the right.
- Increases total list length by 1.

2.7 insert() Walkthrough Step-by-Step

```
L = [10, 20, 30]
L.insert(1, 40)
print(L) # Output: [10, 40, 20, 30]
L.insert(0, "abc")
print(L) # Output: ['abc', 10, 40, 20, 30]
a = "xyz"
L.insert(3, a)
print(L) # Output: ['abc', 10, 40, 'xyz', 20, 30]
L.insert(5, ["a", "b"])
print(L) # Output: ['abc', 10, 40, 'xyz', 20, ['a', 'b'], 30]
```

2.8 The `extend()` Function

Definition & Syntax

Syntax: `list.extend(iterable_value)`

- Used to append **multiple values at once**.
- Unpacks the iterable object (e.g., list) and adds each element individually.

2.9 extend() Code Examples

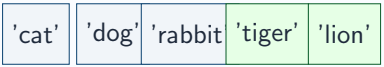
```
# Example 1: Extending numbers
n = [1, 2, 3, 4]
m = [5]
n.extend(m)
print(n)
# Output: [1, 2, 3, 4, 5]
# Example 2: Merging lists without nesting
pets = ['cat', 'dog', 'rabbit']
wild = ['tiger', 'lion']
pets.extend(wild)
print(pets)
# Output: ['cat', 'dog', 'rabbit', 'tiger', 'lion']
```

2.10 Comparative Illustration: `append()` vs `extend()`

pets.append(wild)



pets.extend(wild)



Crucial Difference

`append()` adds the list as a **single nested element**. `extend()` **flattens** the iterable and appends all items individually.

2.11 The `sort()` Method

Definition & Features

Sorts elements of a list in-place (modifies the original list directly).

- By default, sorts in **ascending order**.
- Can sort in **descending order** using `reverse=True`.
- Requires elements to be of **compatible data types**.

2.12 sort() Code Demonstration

```
city = ["Delhi", "Mumbai", "Kolkata", "Chennai"]
# Ascending Sort (Default)
city.sort()
print(city)
# Output: ['Chennai', 'Delhi', 'Kolkata', 'Mumbai']
# Descending Sort
city.sort(reverse=True)
print(city)
# Output: ['Mumbai', 'Kolkata', 'Delhi', 'Chennai']
```

2.13 Searching Values in a List

Searching determines whether an element exists and locates its position.

1. **Counting Occurrences**: Frequency of a specific item.
2. **Linear Search for Index**: Locating position using loops.

2.14 Search Example 1: Occurrence Counter

```
A = [10, 23, 45, 10, 12, 45, 10, 56]
s = int(input("Enter value you want to search: "))
count = 0
for i in A:
    if i == s:
        count += 1
print(s, "occurs", count, "times")
```

Logic

Iterates through each element in list `A`, incrementing `count` whenever a match is detected.

2.15 Search Example 2: Linear Search with Index

```
flowers = ["Rose", "Lily", "Lotus", "Sunflower"]
s = input("Input a name of a flower: ")
for i in range(len(flowers)):
    if s == flowers[i]:
        print(s, "found at index number", i)
        break
else:
    print("Sorry! Not Found")
```

for-else Construct

The `else` block executes only if the loop finishes normally without hitting a `break` statement.

3.1 Introduction to Tuples

What is a Tuple?

A **Tuple** is an ordered sequence of values enclosed in **parentheses** `()`.

- Indices start at `0`.
- **Immutable**: Cannot be changed, modified, or updated after creation.
- Can store heterogeneous elements (integers, strings, lists, tuples).

3.2 Key Features of Tuples

Core Characteristics

- **Immutable**: Elements cannot be modified or replaced.
- **Fixed Length**: Cannot grow or shrink dynamically.
- **Enclosure**: Parentheses `()`.

Performance Features

- **Faster Access**: Read-only nature makes lookup efficient.
- **Heterogeneous**: Holds mixed datatypes.
- **Safe Data**: Prevents accidental modifications.

3.3 Tuple Creation Syntaxes

```
# Heterogeneous / Dissimilar Items
tup1 = ('sst', 'chemistry', 2010, 2022)
# Homogeneous Items
tup2 = (1, 2, 3, 4, 5)
tup3 = (1, 2.5, 3.6, 4, 5)
```

3.4 Single Element Tuple: A Common Pitfall

Important Warning

A single value inside parentheses without a comma is treated as a standard primitive type, **NOT** a tuple!

```
t1 = (50) # Type is int!
print(type(t1)) # Output: <class 'int'>
t2 = (50,) # Type is tuple!
print(type(t2)) # Output: <class 'tuple'>
```

3.5 Program 1: Creating Various Tuples

```
# Homogeneous data elements
tuple1 = (3, 4, 6, 7)
print(tuple1)
# Heterogeneous data elements
tuple2 = (3, "Orange", 15.5)
print(tuple2)
# Nested tuple
tuple3 = (162, [7, "Orange", 15.5], (2, "Orange Education", 70.5))
print(tuple3)
```

3.6 Output Analysis of Program 1

Program 1 Output

```
(3, 4, 6, 7)
(3, 'Orange', 15.5)
(162, [7, 'Orange', 15.5], (2, 'Orange Education', 70.5))
```

- `tuple1`: Standard integer tuple.
- `tuple2`: Mixed integer, string, and float.
- `tuple3`: Contains integer, nested list, and nested tuple.

3.7 Special Creation Methods for Tuples

- **Empty Tuple**: `tup = tuple()` or `tup = ()`
- **Long Tuple**: `tup = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12)`
- **Nested Tuple**: `tup = (1, 2, (3, 4))`
- **From Sequence via** `tuple()`:

```
1 tup = tuple('orange')
2 print(tup)
3 # Output: ('o', 'r', 'a', 'n', 'g', 'e')
```

3.8 Program 2: Accessing Tuple Elements

```
1 tup1 = ('sst', 'chemistry', 2010, 2022)
2 tup2 = (1, 2, 3, 4, 5)
3 print(tup1[1]) # Output: chemistry
4 print(tup2[3]) # Output: 4
```

Access Syntax

Like lists, access tuple items via bracket notation `tup[index]`.

3.9 Deleting a Tuple

Immutability Constraint

Individual elements of a tuple **cannot** be deleted or popped!

- However, the **entire tuple** object can be removed from memory using the `del` keyword.

3.10 Program 3: Deleting a Tuple

```
tuple = (162, [7, 'Orange', 15.5], (2, 'Orange Education', 70.5))
del tuple
# Trying to print 'tuple' now will raise NameError!
print(tuple)
```

Expected Output

```
NameError: name 'tuple' is not defined.
```

4.1 Detailed Comparison: List vs Tuple

Feature	List	Tuple
Sequence Type	Ordered sequence	Ordered collection
Enclosure	Square brackets []	Parentheses ()
Mutability	Mutable (elements can change)	Immutable (elements cannot change)
Length	Variable / Dynamic	Fixed length
Data Types	Homogeneous / Heterogeneous	Homogeneous / Heterogeneous

4.2 Program 4: Printing Entire Tuple

```
1 tup1 = ('sst', 'chemistry', 2010, 2022)
2 print(tup1)
```

Output

```
('sst', 'chemistry', 2010, 2022)
```

4.3 Tuple Built-in Methods

Since tuples are immutable, they only support non-modifying methods:

1. `count(value)`: Returns the number of occurrences of `value`.
2. `index(value)`: Returns the first index position where `value` appears.

4.4 Program 5: count() and index()

```
tup = (2, 5, 7, 8, 2, 1)
print(tup.count(2)) # Counts occurrences of 2
# Output: 2
print(tup.index(7)) # Returns index of element 7
# Output: 2
```

4.5 Converting List to Tuple and Vice-versa

To modify a tuple, a common workaround is:

1. Convert Tuple $\rightarrow$ List using `list()`.
2. Perform modifications on the list.
3. Convert List $\rightarrow$ Tuple back using `tuple()`.

4.6 Type Conversion Mechanics

```
# List to Tuple
L = [1, 2, 3, 4]
T = tuple(L)
print(type(T)) # Output: <class 'tuple'>
# Tuple to List
L2 = list(T)
print(type(L2)) # Output: <class 'list'>
```

4.7 Program 6: Correcting Tuple Elements

Problem Statement

A tuple `tup` stores `(2, 5, 7, 8, 2, 1)`. By mistake, its last element is typed wrong. Write a program to correct its last element as `11`.

4.8 Solution Code for Program 6

```python
tup = (2, 5, 7, 8, 2, 1)
# Step 1: Convert to list
tup1 = list(tup)
# Step 2: Modify last element
tup1[-1] = 11
# Step 3: Convert back to tuple
tup = tuple(tup1)
print("Modified tuple:", tup)
# Output: Modified tuple: (2, 5, 7, 8, 2, 11)
```

5.1 Summary: List Creation Summary Card

List Creation Checklist

- Empty: `[]`
- Elements: `[1, 2, "a"]`
- Multiplication: `[0] * 5`
- Constructor: `list("abc")`

5.2 Summary: Indexing Cheatsheet

Indexing Rules

- First element: `L[0]` or `L[-len(L)]`
- Last element: `L[-1]` or `L[len(L)-1]`
- Out of bounds index raises `IndexError`.

5.3 Summary: List Modification Functions

Adding Methods

- `append(x)`: Adds `x` to end.
- `insert(i, x)`: Adds `x` at index `i`.
- `extend(L)`: Merges list `L`.

5.4 Summary: Tuple Essentials

Tuple Rules

- Single item requires comma: `(x,)`
- Immutable: Cannot reassign elements.
- Methods: `count()`, `index()`.

5.5 Practice Quiz 1

Question 1

What is the result of `len([1, [2, 3], 4])`?

Answer: 3. The nested list `[2, 3]` is counted as a single element.

5.6 Practice Quiz 2

Question 2

What happens when executing `t = (5); type(t)`?

Answer: It evaluates to `<class 'int'>` because no trailing comma was provided.

5.7 Practice Quiz 3

Question 3

How do `append([1,2])` and `extend([1,2])` differ?

Answer: `append` adds `[1,2]` as a single nested list. `extend` unpacks and appends `1` and `2` individually.

5.8 Best Practices Guidelines

- Use **Lists** when data needs to change dynamically (e.g., user inputs, updating records).
- Use **Tuples** for fixed read-only data (e.g., coordinates, constant configuration settings).

5.9 Common Pitfalls to Avoid

- **IndexError**: Accessing an index outside range.
- **TypeError**: Trying to mutate a tuple element.
- **Single-item Tuple Confusion**: Forgetting trailing comma `(val,)`.

5.10 Final Review & Conclusion

Key Concepts Mastered

- List creation, indexing, slicing, and mutation.
- Array operations: `append()`, `insert()`, `extend()`, `sort()`.
- Tuple immutability, methods, and conversion patterns.

Thank You!

Questions & Discussion